

ECE 4502/6502 & CS 6501: Machine Learning on Graphs

Lecture 11. Network Embedding II

Instructor: Jundong Li
Spring 2025, University of Virginia

Random Walk Approaches

- Perozzi et al. 2014. [DeepWalk: Online Learning of Social Representations](#). *KDD*.
- Grover et al. 2016. [node2vec: Scalable Feature Learning for Networks](#). *KDD*.

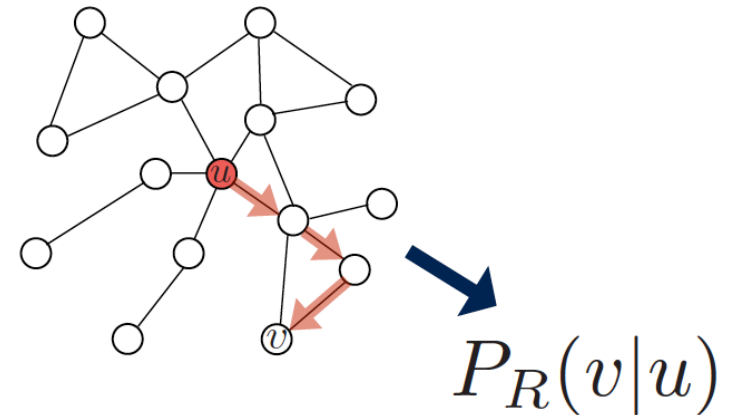
Random-walk Embeddings

$$\mathbf{z}_u^\top \mathbf{z}_v \approx$$

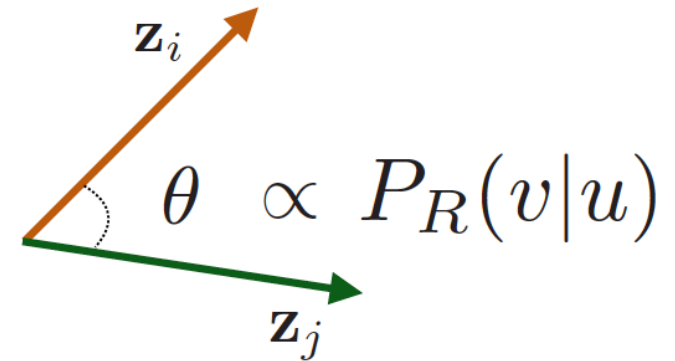
probability that u and v co-occur on a random walk over the network

Random-walk Embeddings

1. Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R .



2. Optimize embeddings to encode these random walk statistics.



Why Random Walks?

- 1. Expressivity:** Flexible stochastic definition of node similarity that incorporates both local and higher-order neighborhood information
- 2. Efficiency:** Do not need to consider all node pairs when training; only need to consider pairs that co-occur on random walks

Random Walk Optimization

1. Run short random walks starting from each node on the graph using some strategy R
2. For each node u collect $N_R(u)$, the multiset of nodes visited on random walks starting from u
3. Optimize embeddings to according to:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

* $N_R(u)$ can have repeat elements since nodes can be visited multiple times on random walks.

Random Walk Optimization

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v | \mathbf{z}_u))$$

- **Intuition:** Optimize embeddings to maximize likelihood of random walk co-occurrences.
- **Parameterize** $P(v | \mathbf{z}_u)$ **using softmax:**

$$P(v | \mathbf{z}_u) = \frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)}$$

Random Walk Optimization

Putting things together:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

sum over all nodes u

sum over nodes v seen on random walks starting from u

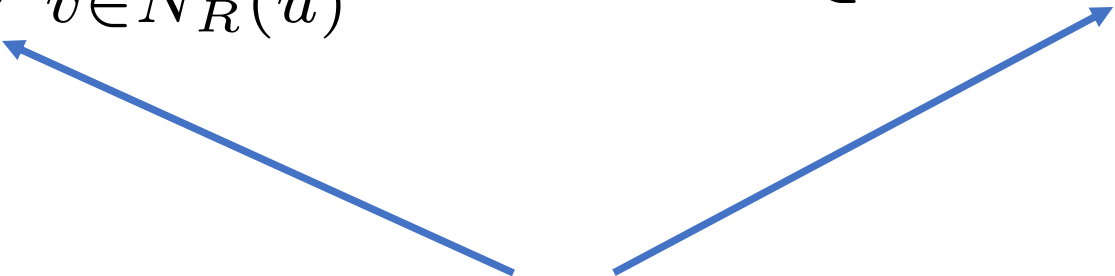
predicted probability of u and v co-occurring on random walk

Optimizing random walk embeddings =

Finding embeddings $\mathbf{z}_u$ that minimize $\mathcal{L}$

Random Walk Optimization

- **But doing this naively is too expensive!!**

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$


Nested sum over nodes gives
 $O(|V|^2)$ complexity!!

Random Walk Optimization

- **But doing this naively is too expensive!!**

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

The normalization term from the softmax is the culprit... can we approximate it?

Negative Sampling

Solution: Negative sampling

$$\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

$$\approx \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \sum_{i=1}^k \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_{n_i})), n_i \sim P_V$$

sigmoid function

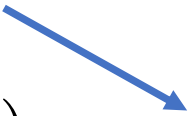
random distribution
over all nodes

i.e., instead of normalizing w.r.t. all nodes, just normalize against k random “**negative samples**”

Negative Sampling Cont.

$$\log \left(\frac{\exp(\mathbf{z}_u^\top \mathbf{z}_v)}{\sum_{n \in V} \exp(\mathbf{z}_u^\top \mathbf{z}_n)} \right)$$

random distribution over all nodes


$$\approx \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_v)) - \sum_{i=1}^k \log(\sigma(\mathbf{z}_u^\top \mathbf{z}_{n_i})), n_i \sim P_V$$

- Sample negative nodes proportional to degree
- Two considerations for k (# negative samples):
 1. Higher k gives more robust estimates
 2. Higher k corresponds to higher prior on negative events

Random Walks: Stepping Back

1. Run short random walks starting from each node on the graph using some strategy R .
2. For each node u collect $N_R(u)$, the multiset of nodes visited on random walks starting from u .
3. Optimize embeddings to according to:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

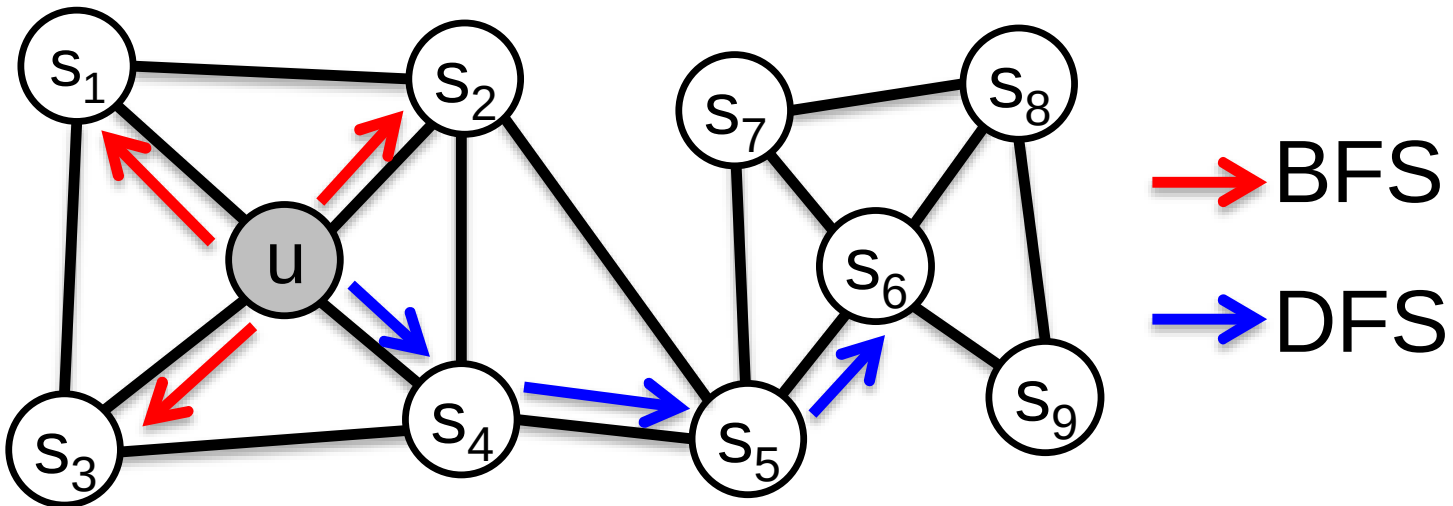
We can efficiently approximate this using
negative sampling!

How Should we Randomly Walk?

- So far we have described how to optimize embeddings given random walk statistics
- **What strategies should we use to run these random walks?**
 - Simplest idea: Just run fixed-length, unbiased random walks starting from each node (i.e., [DeepWalk from Perozzi et al., 2014](#)).
 - But can we do better?

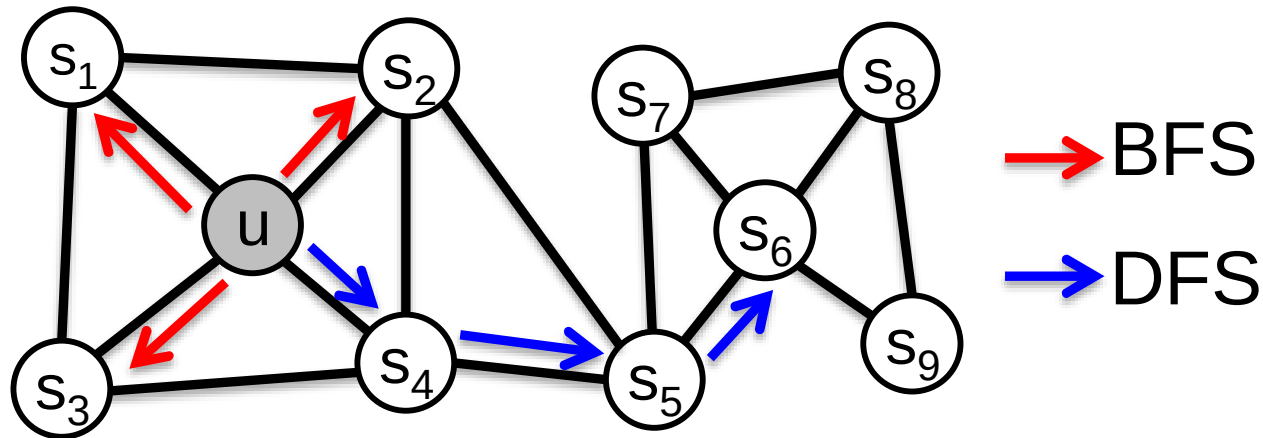
node2vec: Biased Walks

- **Idea:** use flexible, biased random walks that can trade off between **local** and **global** views of the network ([Grover and Leskovec, 2016](#)).



node2vec: Biased Walks

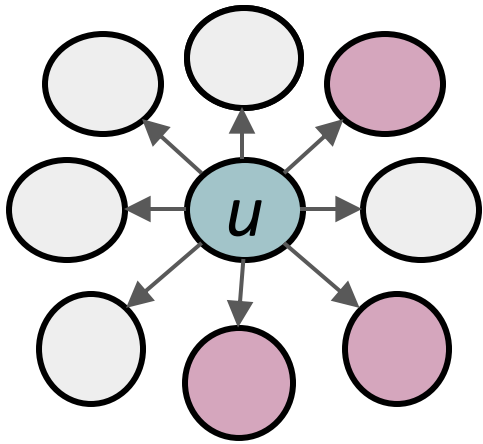
- Two classic strategies to define a neighborhood $N_R(u)$ of a given node u :



$N_{BFS}(u) = \{s_1, s_2, s_3\}$ Local microscopic view

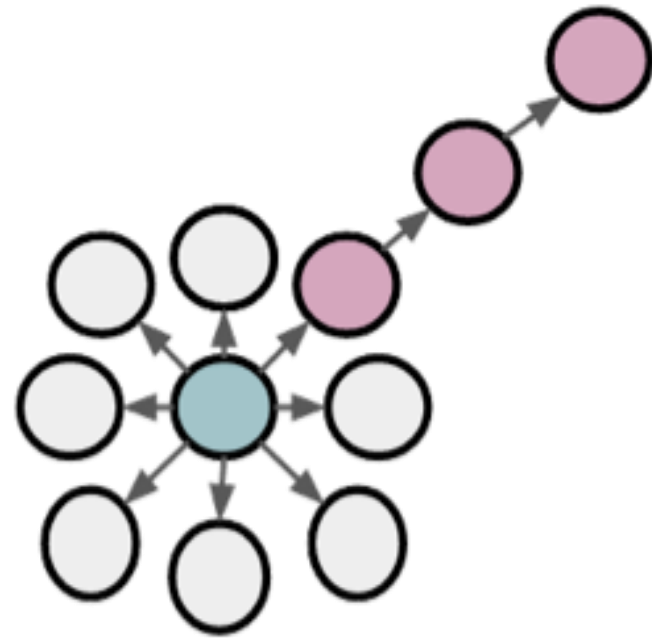
$N_{DFS}(u) = \{s_4, s_5, s_6\}$ Global macroscopic view

BFS vs. DFS



BFS:

Micro-view of
neighbourhood



DFS:

Macro-view of
neighbourhood

Interpolating BFS and DFS

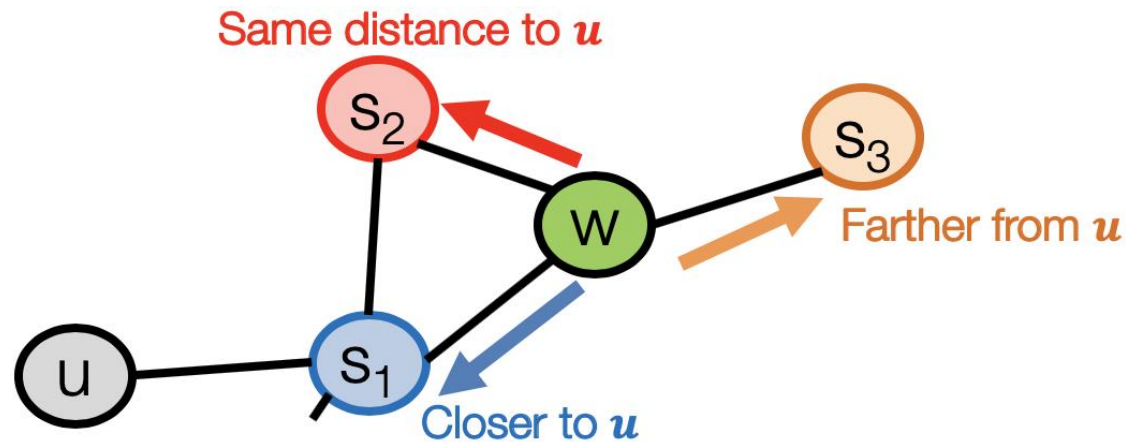
Biased random walk R that given a node u generates neighborhood $N_R(u)$

- Two parameters:
 - Return parameter p :
 - Return back to the previous node (BFS)
 - In-out parameter q :
 - Moving outwards (DFS)

Biased Random Walks

Biased 2nd-order random walks explore network neighborhoods:

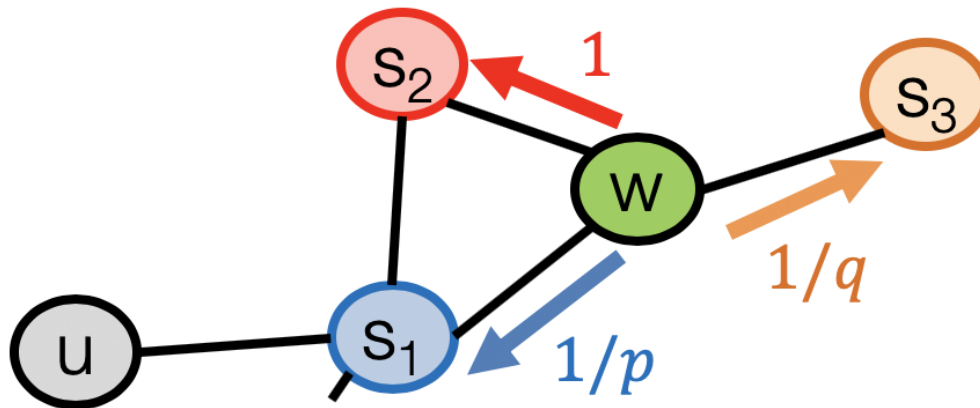
- Rnd. walk started at u and is now at w
- **Insight:** Neighbors of w can only be:



Idea: Remember where that walk came from

Biased Random Walks

- Walker is at w . Where to go next?

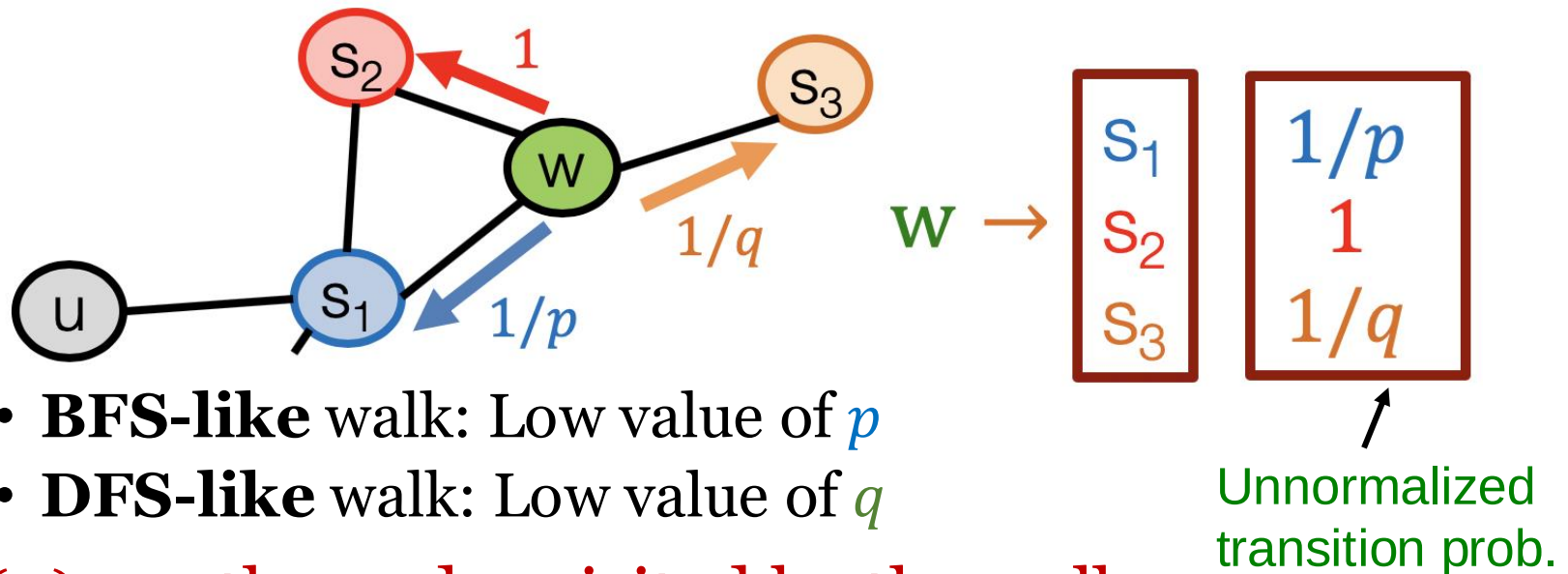


$1/p, 1/q, 1$ are
unnormalized
probabilities

- p, q model transition probabilities
 - p ... return parameter
 - q ... “walk away” parameter

Biased Random Walks

- Walker is at w . Where to go next?

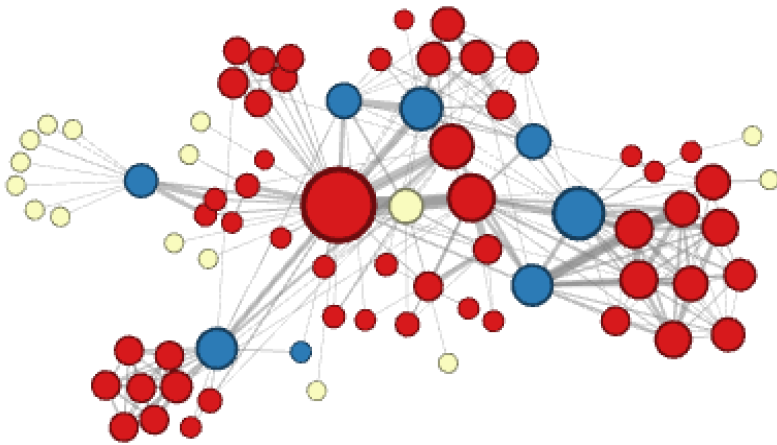


- **BFS-like** walk: Low value of p
- **DFS-like** walk: Low value of q

$N_S(u)$ are the nodes visited by the walker

Experiments: Micro vs. Macro

Interactions of characters in a novel:



$p=1, q=2$

Microscopic view of the
network neighbourhood



$p=1, q=0.5$

Macroscopic view of the
network neighbourhood

Other Random Walk Ideas

- Different kinds of biased random walks:
 - Based on node attributes (Dong et al., 2017).
 - Based on learned weights (Abu-El-Haija et al., 2017)
- Alternative optimization schemes:
 - Directly optimize based on 1-hop and 2-hop random walk probabilities (as in LINE from Tang et al. 2015).
- Network preprocessing techniques:
 - Run random walks on modified versions of the original network (e.g., Ribeiro et al. 2017's struct2vec, Chen et al. 2016's HARP).

Summary So Far

- **Basic idea:** Embed nodes so that distances in embedding space reflect node similarities in the original network.
- Different notions of node similarity:
 - Adjacency-based (i.e., similar if connected)
 - Multi-hop similarity definitions.
 - Random walk approaches

Summary So Far

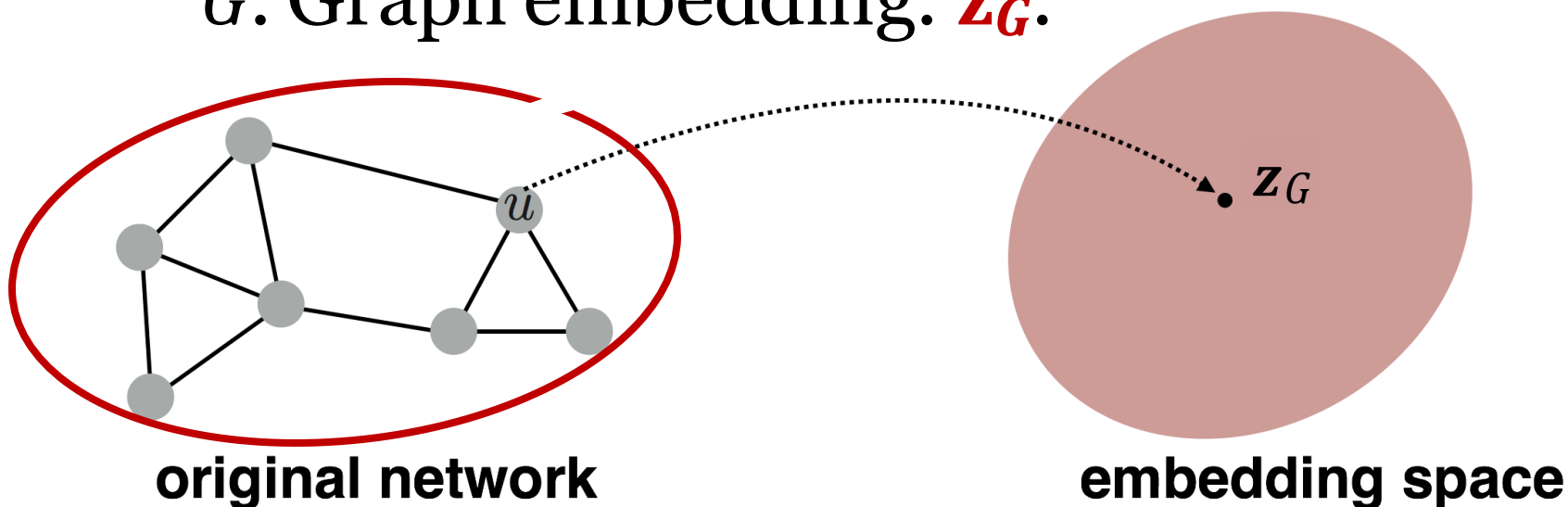
- **So what method should I use..?**
- No one method wins in all cases....
 - E.g., node2vec performs better on node classification while multi-hop methods performs better on link prediction ([Goyal and Ferrara, 2017 survey](#)).
- Random walk approaches are generally more efficient (i.e., $O(|E|)$ vs. $O(|V|^2)$)
- **In general:** Must choose definition of node similarity that matches application!

Embedding Entire Graphs

Embedding Entire Graphs

- **Goal**

- Embed a subgraph or an entire graph G . Graph embedding: $\mathbf{z}_G$.



- **Example tasks:**

- Classifying toxic vs. non-toxic molecules
- Identifying anomalous graphs

Approach 1

Simple (but effective) approach 1:

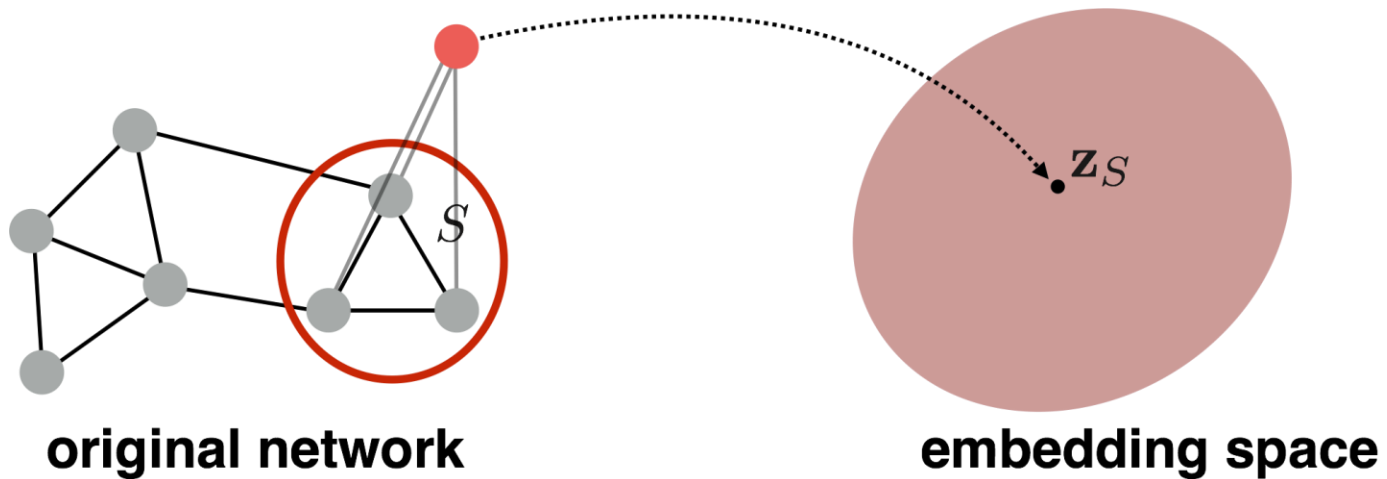
- Run a standard graph embedding technique *on* the (sub)graph G .
- Then just sum (or average) the node embeddings in the (sub)graph G .

$$\mathbf{z}_G = \sum_{v \in G} \mathbf{z}_v$$

- Used by [Duvenaud et al.](#) to classify molecules based on their graph structure

Approach 2

- **Approach 2:** Introduce a “**virtual node**” to represent the (sub)graph and run a standard graph embedding technique



- Proposed by [Li et al.](#) as a general technique for subgraph embedding

Summary

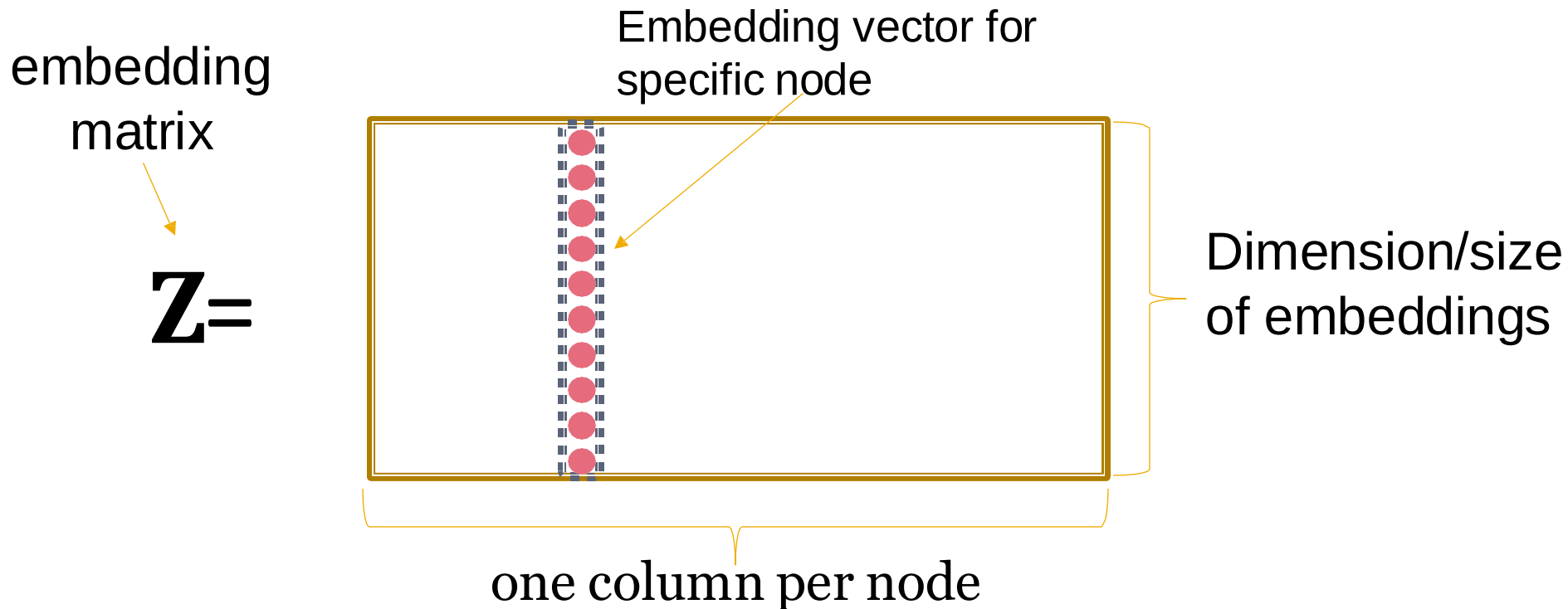
We discussed 2 ideas to graph embeddings:

- **Approach 1:** Embed nodes and sum/avg them
- **Approach 2:** Create super-node that spans the (sub) graph and then embed that node.

Matrix Factorization vs. Node Embedding

Embeddings & Matrix Factorization

- **Recall:** encoder as an embedding lookup



Objective: maximize $\mathbf{z}_u^T \mathbf{z}_v$ for node pairs (u, v) that are **similar**

Embeddings & Matrix Factorization

- Adjacency based similarity

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}\|^2$$

- Multi-hop similarity

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \|\mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v}^k\|^2$$

- Observation

- Objective: $\min \|\tilde{A} - Z^T Z\|_2$
- Conclusion: Inner product decoder with node similarity defined by $\tilde{A}$ is equivalent to matrix factorization

Random Walk-based Similarity

- **DeepWalk** and **node2vec** have a more complex **node similarity** definition based on random walks
- **DeepWalk** is equivalent to matrix
 - Factorization of the following complex matrix expression:

$$\log \left(\text{vol}(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b$$

Random Walk-based Similarity

Volume of graph

$$\text{vol}(G) = \sum_i \sum_j A_{i,j}$$

Diagonal matrix D

$$D_{u,u} = \text{deg}(u)$$

$$\log \left(\text{vol}(G) \left(\frac{1}{T} \sum_{r=1}^T (D^{-1}A)^r \right) D^{-1} \right) - \log b$$

context window size

Power of normalized
adjacency matrix

Number of
negative samples

- **Node2vec** can also be formulated as a matrix factorization (albeit a more complex matrix)
- Refer to the paper for more details:

How to Use Embeddings

- **How to use embeddings $\mathbf{z}_i$ of nodes:**
 - **Clustering/community detection:** Cluster points $\mathbf{z}_i$
 - **Node classification:** Predict label of node i based on $\mathbf{z}_i$
 - **Link prediction:** Predict edge (i, j) based on $(\mathbf{z}_i, \mathbf{z}_j)$
 - Where we can: concatenate, avg, product, or take a difference between the embeddings:
 - Concatenate: $f(\mathbf{z}_i, \mathbf{z}_j) = g([\mathbf{z}_i, \mathbf{z}_j])$
 - Hadamard: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\mathbf{z}_i * \mathbf{z}_j)$ (per coordinate product)
 - Sum/Avg: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\mathbf{z}_i + \mathbf{z}_j)$
 - Distance: $f(\mathbf{z}_i, \mathbf{z}_j) = g(\|\mathbf{z}_i - \mathbf{z}_j\|_2)$
 - **Graph classification:** Graph embedding $\mathbf{z}_G$ via aggregating node embeddings or virtual-node.
 - Predict label based on graph embedding $\mathbf{z}_G$.

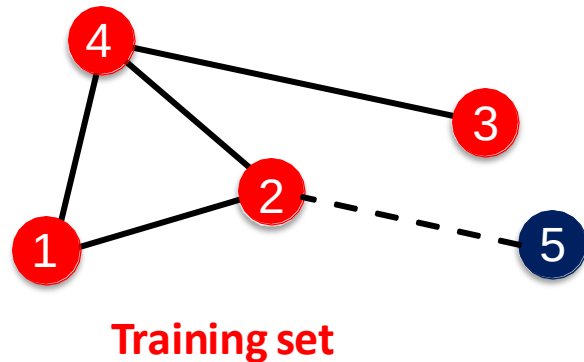
Summary

- Graph representation learning
 - Learn node/graph embedding without feature engineering
- Encoder-decoder framework:
 - **Encoder: embedding lookup**
 - **Decoder: predict score based on embedding to match node similarity**
- Node similarity measure: (biased) random walk
 - **Examples: DeepWalk, Node2Vec**
- Extension to Graph embedding: Node embedding aggregation

Limitations

Limitation 1. Transductive (not inductive) method:

- Cannot obtain embeddings for nodes not in the training set
- Cannot apply to new graphs
 - If you apply DeepWalk to the same graph multiple times, each time you'll get a different embedding each time. Why?

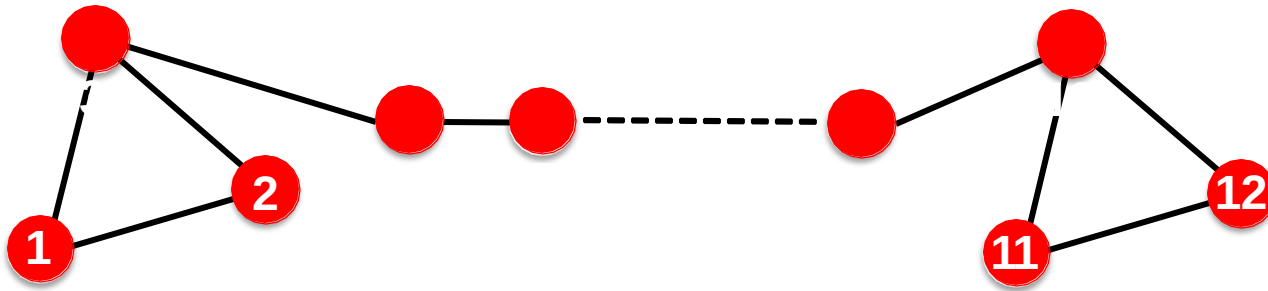


A newly added node 5 at test time
(e.g., new user in a social network)

Cannot compute its embedding
with DeepWalk / node2vec. Need to
recompute all node embeddings.

Limitations

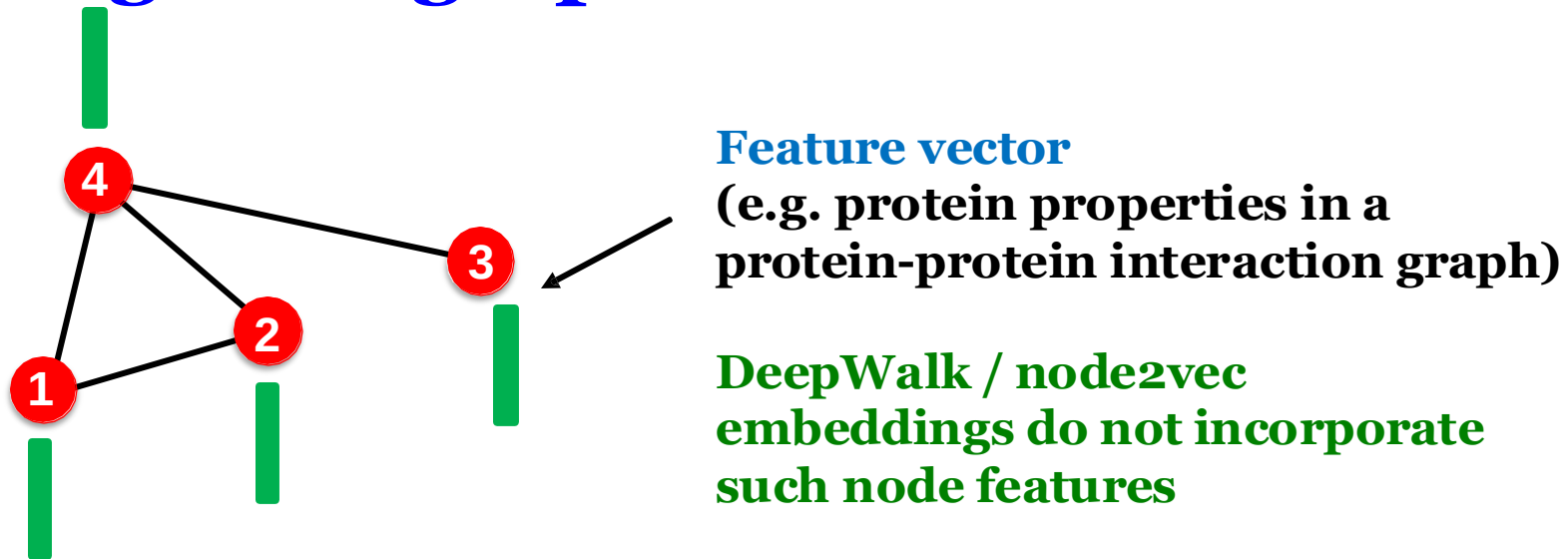
Limitation 2. Cannot capture structural similarity



- Node 1 and 11 are **structurally similar** – part of one triangle, degree 2, ...
- However, they have very **different** embeddings.
 - It's unlikely that a random walk will reach node 11 from node 1.
- **DeepWalk and node2vec do not capture structural similarity.**

Limitations

Limitation 3. Cannot utilize node, edge and graph features



Solution to these limitations: Deep Representation Learning and Graph Neural Networks

(To be covered in depth next)



Questions?